INFORME DE ENTREGA

DATACIUDAD

Implementación de los puntos 1, 2, 3 y 4

Desarrollo de Software II

Docente: John Robert Causimanse

Ingeniero desarrollado y máster en administración de proyectos de innovación

Integrantes

Luis David Rico

Sebastián Reyes

Felipe Torres

Santiago de Cali, Colombia

2026

Tabla de contenido

1. 1. Introducción
2. 2. Contexto del problema y alcance
3. 3. Metodología Scrum y sus fases
4. 4. Historias de usuario y requisitos
5. 5. Diseño técnico y tecnologías
6. 6. Diseño y modelo de base de datos
7. 7. Seguridad y protección de datos
8. 8. DevOps: Git
9. 9. Bitácoras de herramientas
10. 10. Plan de pruebas y criterios de aceptación
11. 11. Riesgos, limitaciones y mejoras
12. 12. Conclusiones
13. 13. Referencias
14. Anexo A. Distribución sugerida de responsabilidades

1. Introducción

El presente informe documenta la entrega correspondiente a la implementación de los puntos 1, 2, 3 y 4 del proyecto Dataciudad, planteado para una empresa encuestadora que presta sus servicios en zonas urbanas y rurales de Cali. La solución propuesta es una aplicación web que centraliza el registro de solicitudes, controla el acceso según el perfil del usuario, permite asociar coordenadas geográficas a cada solicitud y ofrece al perfil enrutador una bitácora diaria de los registros recibidos.

El documento toma como fuente funcional el enunciado académico suministrado para Desarrollo de Software II. El enlace de la rama entrega-1-puntos-1-4 fue identificado, pero su contenido no pudo inspeccionarse en esta sesión. Por tal motivo, las descripciones de tecnologías, estructura de base de datos y DevOps se presentan distinguiendo entre requisitos del enunciado, diseño técnico recomendado y evidencias pendientes de contrastar con el repositorio.

2. Contexto del problema y alcance

La operación descrita recibe solicitudes en cinco puntos de atención y en la oficina principal. La asignación de encuestas se concentra en la oficina principal y actualmente depende de procesos manuales. El enunciado indica que el 30 % de las encuestas se realiza por fuera del plazo institucional de 15 días calendario. También señala errores frecuentes en formularios físicos: barrios o comunas incorrectos, inconsistencias de edad y fecha de nacimiento, preguntas incompletas y ausencia del nombre del encuestador.

La primera entrega aborda únicamente los cuatro requerimientos solicitados: autenticación por roles, captura de solicitudes, georreferenciación y bitácora diaria. No se considera implementada en esta fase la generación automática de rutas, la asignación de cargas, el diligenciamiento digital completo de encuestas ni el dictamen de calidad, salvo que el código del repositorio demuestre lo contrario.

2.1 Objetivo general

Desarrollar y documentar los componentes iniciales de una aplicación web que permita gestionar de forma controlada las solicitudes de encuesta, identificar su ubicación y facilitar su consulta diaria por parte del personal encargado de la planificación.

2.2 Objetivos específicos

- Implementar autenticación de usuarios y autorización basada en cinco perfiles.
- Permitir al perfil ventanilla registrar los datos mínimos de una solicitud.
- Asociar coordenadas geográficas a cada solicitud y conservar su referencia territorial.
- Presentar al perfil enrutador un registro consultable de las solicitudes creadas durante el día.
- Definir una base de datos relacional y un flujo de trabajo que permita evolucionar el sistema por entregas.

3. Metodología Scrum y sus fases

Scrum se utiliza como marco de trabajo iterativo e incremental. Para esta entrega se organiza el trabajo en un Product Backlog, una planificación del Sprint, ejecución diaria, revisión del incremento y retrospectiva. Las ceremonias y roles que se describen a continuación son la forma recomendada de organizar el proyecto; las fechas, asistentes y actas reales deben completarse con los registros del equipo.

3.1 Roles

- Product Owner: representa las necesidades del proceso de encuestas, prioriza el backlog y acepta los criterios de aceptación.
- Scrum Master: facilita las ceremonias, identifica impedimentos y promueve el cumplimiento del marco Scrum.
- Developers: diseñan, implementan, prueban y documentan la aplicación. Para esta entrega, los integrantes del equipo son Luis David Rico, Sebastián Reyes y Felipe Torres.

3.2 Fases y aplicación al proyecto

Fase / evento	Aplicación en Dataciudad	Resultado esperado
Visión y Product Backlog	Convertir el enunciado en épicas e historias priorizadas: acceso, registro, georreferenciación y bitácora.	Backlog priorizado con criterios de aceptación.
Sprint Planning	Seleccionar las historias 1 a 4, estimar esfuerzo, definir tareas y objetivo del Sprint.	Sprint Backlog y objetivo acordado.
Ejecución del Sprint	Diseño de pantallas, modelo de datos, implementación de controladores, validaciones y pruebas.	Incremento funcional integrado.
Daily Scrum	Reunión breve para revisar avance, plan del día e impedimentos.	Coordinación y bloqueos visibles.
Sprint Review	Demostración del login por roles, creación de solicitudes, mapa/coordenadas y consulta diaria.	Retroalimentación y aceptación del incremento.
Sprint Retrospective	Identificar qué funcionó, qué dificultó el trabajo y qué mejorar en el siguiente Sprint.	Acciones de mejora concretas.
Incremento / entrega	Integrar código probado, documentación y evidencias de ejecución.	Versión entregable y verificable.

3.3 Definition of Done (criterio de terminado)

Una historia se considera terminada cuando el código está integrado en la rama de entrega, las validaciones esenciales pasan, no quedan errores bloqueantes conocidos, se han probado los permisos por rol, existe documentación mínima y la funcionalidad puede demostrarse en un entorno reproducible. La evidencia debe incluir el commit o pull request relacionado y, cuando aplique, el resultado de las pruebas o del pipeline.

4. Historias de usuario y requisitos

Los requisitos funcionales se expresan como historias de usuario y se acompañan de criterios de aceptación comprobables. Los requisitos no funcionales se asocian a las historias cuando condicionan su calidad.

HU-01 – Inicio de sesión y perfiles

Como usuario del sistema, quiero iniciar sesión con mis credenciales para acceder únicamente a las funciones correspondientes a mi perfil.

- Criterios: el sistema solicita usuario/correo y contraseña; valida las credenciales; permite los perfiles administrador, ventanilla, encuestador, calidad y enrutador; rechaza credenciales inválidas; restringe rutas y operaciones según el rol; permite cerrar sesión.
- RF asociados: RF-01 autenticación; RF-02 gestión/identificación de roles; RF-03 autorización por perfil; RF-04 cierre de sesión.
- RNF asociados: RNF-01 contraseñas almacenadas mediante hash seguro; RNF-02 sesión protegida; RNF-03 mensajes de error sin revelar si una cuenta existe; RNF-04 interfaz usable y adaptable.

HU-02 – Registro de solicitud en ventanilla

Como usuario de ventanilla, quiero registrar los datos de una persona interesada en una encuesta para dejar constancia de su solicitud y permitir su gestión posterior.

- Criterios: captura nombres, apellidos, barrio, comuna, celular y dirección; valida campos obligatorios y formatos; registra fecha/hora de creación y usuario que atendió; evita guardar registros incompletos; relaciona barrio, comuna y corregimiento con la división territorial de Cali.
- RF asociados: RF-05 crear solicitud; RF-06 validar datos; RF-07 consultar divisiones territoriales; RF-08 registrar fecha y usuario creador.
- RNF asociados: RNF-05 integridad referencial; RNF-06 protección de datos personales; RNF-07 validación tanto en cliente como en servidor.

HU-03 – Georreferenciación de solicitud

Como usuario de ventanilla, quiero ubicar geográficamente la dirección de la solicitud para que el equipo pueda identificar el lugar de la visita y apoyar la planeación territorial.

- Criterios: permite seleccionar o marcar una ubicación en el mapa; guarda latitud y longitud asociadas a la solicitud; valida que las coordenadas estén dentro de rangos geográficos válidos; muestra la ubicación guardada al consultar la solicitud; permite corregirla cuando exista autorización.
- RF asociados: RF-09 asociar coordenadas; RF-10 visualizar ubicación; RF-11 actualizar ubicación con control de permisos.
- RNF asociados: RNF-08 precisión y consistencia de coordenadas; RNF-09 disponibilidad del mapa o mecanismo alterno de captura; RNF-10 informar claramente cuando el servicio cartográfico no esté disponible.

HU-04 – Bitácora diaria para enrutador

Como usuario enrutador, quiero consultar las solicitudes generadas durante el día en todos los puntos de atención para conocer la demanda recibida y preparar la planificación.

- Criterios: muestra solicitudes de la fecha seleccionada; incluye registros de todos los puntos de atención; presenta datos básicos de identificación, dirección, barrio/comuna, fecha y coordenadas cuando existan; permite ordenar o filtrar; el perfil no autorizado no puede acceder a la bitácora.
- RF asociados: RF-12 listar solicitudes del día; RF-13 consolidar puntos de atención; RF-14 filtrar por fecha/punto/territorio; RF-15 controlar acceso por rol.
- RNF asociados: RNF-11 consulta eficiente; RNF-12 trazabilidad de creación; RNF-13 consistencia temporal y zona horaria.

4.1 Requisitos no funcionales transversales

- Seguridad: contraseñas con hash robusto, consultas parametrizadas/ORM, validación de entradas, protección de sesión y principio de mínimo privilegio.
- Privacidad: recolectar únicamente los datos necesarios, limitar la exposición de teléfonos y direcciones, y restringir la consulta según funciones.
- Usabilidad: formularios claros, etiquetas visibles, mensajes de validación comprensibles y diseño adaptable a escritorio y móvil.
- Rendimiento: las consultas de la bitácora deben estar indexadas por fecha de creación y permitir paginación cuando el volumen crezca.
- Mantenibilidad: separación por capas o módulos, convenciones de código, control de versiones y documentación de instalación.
- Disponibilidad y resiliencia: manejo de errores, registro técnico de fallos y comportamiento alternativo ante indisponibilidad del mapa.

5. Diseño técnico y tecnologías

El enunciado académico establece PHP, MySQL, CSS y HTML como tecnologías requeridas o sugeridas, y permite incorporar otras tecnologías para lograr una interfaz adecuada. Como no fue posible inspeccionar los archivos de la rama GitHub, esta sección define el stack de referencia que debe cotejarse con composer.json, package.json, archivos de configuración y código fuente antes de presentarlo como stack efectivamente utilizado.

Tecnología / herramienta	Uso en la solución	Estado de verificación
PHP	Lógica del servidor, validación, autenticación y acceso a datos, si corresponde al código del proyecto.	Requerida/sugerida por el enunciado; cotejar con repositorio.
MySQL	Persistencia relacional de usuarios, roles, solicitudes y divisiones territoriales.	Requerida/sugerida por el enunciado; cotejar configuración.
HTML5	Estructura semántica de formularios, vistas y tablas.	Requerida/sugerida por el enunciado.
CSS3	Presentación visual, diseño adaptable y estados de formularios.	Requerida/sugerida por el enunciado.
JavaScript	Interacciones del mapa, validaciones de interfaz y actualización dinámica, si se utiliza.	Propuesta; verificar dependencias.
Librería/API cartográfica	Visualización de mapa y selección de coordenadas (por ejemplo, Leaflet con OpenStreetMap).	Alternativa de diseño; confirmar implementación real.
Git y GitHub	Control de versiones, ramas, commits y colaboración.	Repositorio GitHub suministrado; verificar historial.

5.1 Arquitectura lógica propuesta

La aplicación se organiza en una capa de presentación (vistas y formularios), una capa de aplicación (controladores, autenticación, autorización y validaciones), una capa de dominio (reglas de solicitudes y georreferenciación) y una capa de persistencia (repositorios/consultas hacia MySQL). Los servicios externos de mapas se consumen desde la interfaz o desde un servicio dedicado, sin convertirlos en fuente única de la dirección registrada.

- Presentación: login, formulario de solicitud, selector de ubicación y bitácora.
- Aplicación: autenticación, middleware de roles, validación de datos y operaciones CRUD.
- Dominio: reglas de creación, asociación territorial y trazabilidad.
- Persistencia: tablas relacionales con claves foráneas, restricciones e índices.

6. Diseño y modelo de base de datos

El siguiente modelo es una propuesta lógica coherente con los cuatro requerimientos. Debe ajustarse a los nombres y columnas reales del esquema que se encuentre en el repositorio. Se recomienda que la ubicación geográfica sea opcional al crear la solicitud, pero que el estado de georreferenciación quede explícito para identificar registros pendientes.

Tabla	Propósito	Campos principales propuestos
roles	Catálogo de perfiles.	id_rol (PK), nombre, descripcion
usuarios	Cuentas de acceso al sistema.	id_usuario (PK), id_rol (FK), nombre, correo, password_hash, activo, creado_en
puntos_atencion	Sedes que reciben solicitudes.	id_punto (PK), nombre, comuna, direccion, activo
corregimientos	División territorial rural.	id_corregimiento (PK), nombre
comunas	División territorial urbana.	id_comuna (PK), numero, nombre
barrios	Barrios asociados a una comuna o corregimiento.	id_barrio (PK), nombre, id_comuna (FK, nullable), id_corregimiento (FK, nullable)
solicitudes	Registro principal de cada solicitud.	id_solicitud (PK), nombres, apellidos, celular, direccion, id_barrio (FK), id_punto (FK), creado_por (FK usuarios), creado_en, estado
geolocalizaciones	Coordenadas asociadas a una solicitud.	id_geolocalizacion (PK), id_solicitud (FK/UNIQUE), latitud, longitud, precision_m, actualizado_por (FK), actualizado_en
bitacora_eventos	Trazabilidad de acciones relevantes.	id_evento (PK), id_usuario (FK), id_solicitud (FK), accion, detalle, fecha_hora

6.1 Relaciones y cardinalidades

- roles 1:N usuarios: un rol puede estar asignado a muchos usuarios; cada usuario tiene un rol principal.
- puntos_atencion 1:N solicitudes: una sede puede registrar muchas solicitudes.
- usuarios 1:N solicitudes: un usuario de ventanilla puede crear múltiples solicitudes.
- comunas 1:N barrios y corregimientos 1:N barrios: cada barrio pertenece a la división territorial que corresponda.

- barrios 1:N solicitudes: un barrio puede aparecer en muchas solicitudes.
- solicitudes 1:0..1 geolocalizaciones: una solicitud puede tener una ubicación geográfica registrada.
- usuarios 1:N bitacora_eventos y solicitudes 1:N bitacora_eventos: las acciones quedan asociadas al usuario y, cuando aplique, a la solicitud.

6.2 Reglas de integridad

- Correo o identificador de acceso único para cada usuario.
- Latitud entre -90 y 90; longitud entre -180 y 180.
- No permitir barrio sin referencia territorial válida.
- Registrar automáticamente creador y fecha/hora de la solicitud.
- La bitácora debe conservar eventos; no se recomienda permitir su edición ordinaria.
- Aplicar índices a solicitudes.creado_en, solicitudes.id_punto y solicitudes.id_barrio para acelerar filtros.

6.3 Diagrama Entidad–Relación

El diagrama incluido representa las entidades y relaciones propuestas. No debe confundirse con una exportación del esquema físico actual del repositorio.

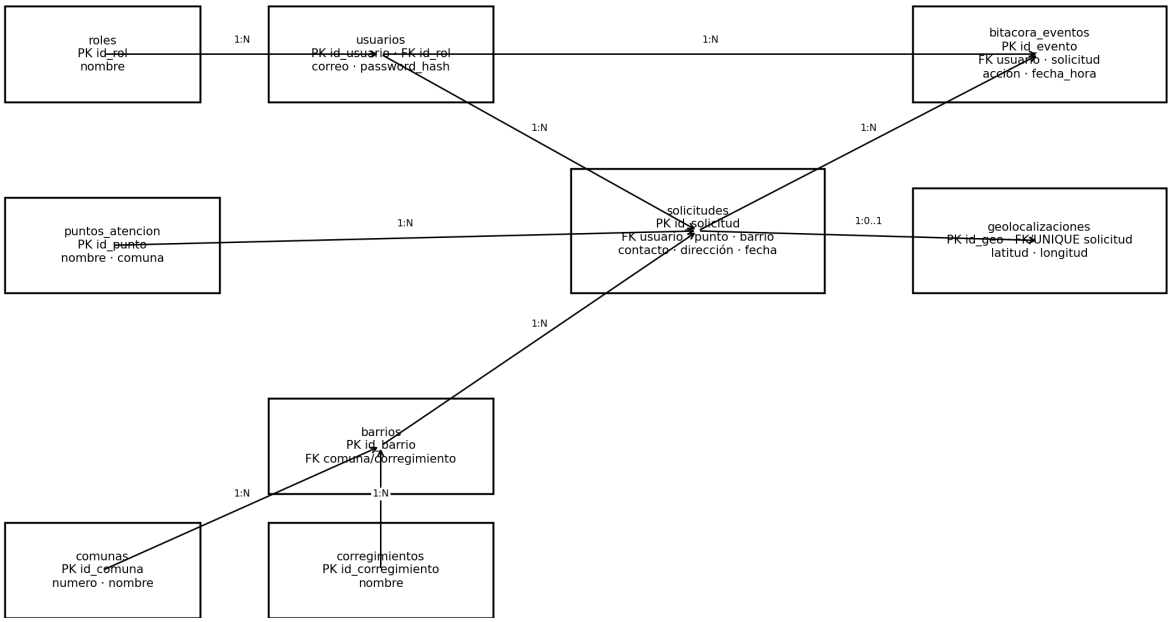


Figura 1. Diagrama Entidad–Relación lógico propuesto para la entrega.

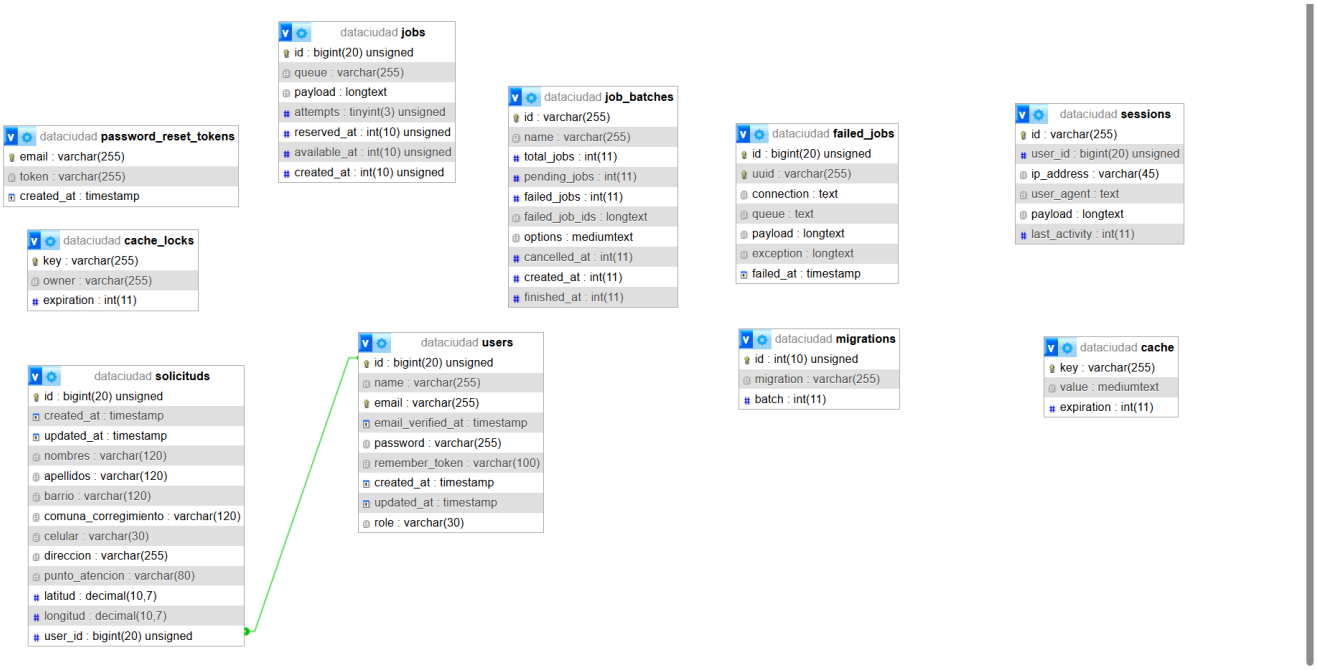


Figura 2. Diagrama Entidad–Relación lógico por phpMyAdmin.

7. Seguridad y protección de datos

- Aplicar control de acceso del lado del servidor; ocultar botones en la interfaz no reemplaza la autorización.
- Almacenar contraseñas con un algoritmo de hashing apropiado (por ejemplo, Argon2id o bcrypt), nunca en texto plano.
- Validar y normalizar nombres, teléfonos, direcciones y parámetros recibidos.
- Usar consultas preparadas o un ORM para reducir el riesgo de inyección SQL.
- Evitar mostrar teléfonos y direcciones completas a perfiles que no las necesitan.
- Usar HTTPS en despliegues y mantener secretos en variables de entorno.
- Registrar accesos y cambios críticos sin incluir contraseñas ni secretos en logs.

Los datos personales de contacto y domicilio deben tratarse con acceso limitado, finalidad definida y medidas de seguridad apropiadas. La política de retención y el aviso de privacidad deben ser definidos por la entidad responsable.

8. DevOps: Git, CI/CD, Docker y Jira

DevOps integra desarrollo y operación mediante colaboración, automatización, trazabilidad y entregas reproducibles. El enunciado solicita evidenciar herramientas como Jenkins o GitHub Actions, Git, Docker y Jira. La existencia y uso real de cada herramienta debe demostrarse con capturas, enlaces o archivos de configuración.

8.1 Git y GitHub

- Uso esperado: repositorio remoto, ramas por funcionalidad/entrega, commits descriptivos, revisión de cambios mediante pull request y etiquetas para identificar versiones.
- Evidencia a anexar: URL del repositorio y rama, historial de commits con autor y fecha, pull requests y captura del estado de la rama.

Comandos de referencia: git status; git branch; git log --oneline --decorate; git remote -v. Ejecutarlos en el repositorio real y anexar la salida obtenida.

8.2 GitHub Actions o Jenkins

- Pipeline recomendado: checkout del código, instalación de dependencias, análisis estático/lint, pruebas automatizadas y construcción del artefacto o imagen.
- Evidencia a anexar: archivo .github/workflows/*.yml o Jenkinsfile, ejecución identificable, fecha, commit asociado y resultado de cada etapa.

No se afirma que exista un pipeline configurado. Si no está implementado, reportarlo como actividad pendiente o mejora propuesta.

8.3 Docker

- Objetivo: reproducir el entorno de ejecución y aislar aplicación y base de datos.
- Evidencia a anexar: Dockerfile, docker-compose.yml/compose.yaml, salida de docker compose ps y logs de arranque.

No se afirma que el proyecto tenga contenedores. Si no existen Dockerfile o archivo Compose en la rama, describir Docker como propuesta y no como herramienta utilizada.

8.4 Jira

- Uso esperado: registrar las historias HU-01 a HU-04 como elementos del backlog, descomponerlas en tareas, asignar responsables, estimar puntos y moverlas por estados.
- Evidencia a anexar: tablero del Sprint, detalle de historias, responsables, fechas, estados y burndown si se trabajó con él.

Si el equipo gestionó el trabajo mediante otra herramienta o directamente en GitHub, debe indicarlo con transparencia y no afirmar uso de Jira sin evidencia.

9. Bitácoras de herramientas

Se presentan dos formatos de bitácora listos para completar. Son plantillas de documentación, no registros históricos generados automáticamente. Para convertirlos en evidencia de uso, completar cada fila con datos reales y adjuntar capturas o enlaces verificables.

9.1 Bitácora de Git

Fecha/hora	Integrante	Rama	Commit (hash)	Descripción del cambio	Evidencia
10/nov/2026	Luis David Rico	main	Commit inicial: Estructura MVC en Laravel	Inicializacion de todo el proyecto	**7e3b259**
10/nov/2026	Sebastian Reyes	Entrega de puntos 1, 2, 3 y 4	Entrega de puntos 1, 2, 3 y 4	Integracion de los requerimientos 1,2,3,4	**e10552b**

Alternativa: si no se configuró CI/CD, la segunda bitácora puede ser de Docker o Jira, siempre que se haya utilizado efectivamente. En ese caso se debe registrar fecha, usuario, comando/acción, resultado y evidencia.

10. Plan de pruebas y criterios de aceptación

ID	Escenario	Resultado esperado
T-01	Acceso con credenciales válidas para cada uno de los cinco perfiles.	Ingreso exitoso y menú/rutas correspondientes al rol.
T-02	Acceso con contraseña incorrecta.	Acceso rechazado con mensaje genérico.
T-03	Usuario sin rol autorizado intenta abrir módulo restringido.	Acceso denegado en servidor.
T-04	Ventanilla registra solicitud con todos los campos válidos.	Solicitud guardada y asociada a creador, punto y fecha.
T-05	Ventanilla envía formulario con campo obligatorio vacío.	No se guarda; se muestra validación.
T-06	Se selecciona ubicación en el mapa.	Latitud/longitud quedan asociadas a la solicitud.
T-07	Se envían coordenadas fuera de rango.	Sistema rechaza la ubicación.
T-08	Enrutador consulta bitácora del día.	Se listan las solicitudes del día de todos los puntos.
T-09	Usuario no enrutador consulta bitácora restringida.	Acceso denegado.
T-10	Solicitud sin coordenadas.	La bitácora identifica ubicación pendiente sin fallar.

Los resultados de ejecución (aprobado/reprobado), fecha, responsable, entorno y evidencia deben completarse después de ejecutar las pruebas sobre la versión entregada. La tabla anterior define casos de prueba, no certifica que hayan sido ejecutados.

11. Riesgos, limitaciones y mejoras

- Calidad de direcciones: los datos escritos por usuarios pueden contener errores; se recomienda catálogo territorial y validación asistida.
- Conectividad rural: diseñar un mecanismo de captura tolerante a interrupciones o un procedimiento de sincronización posterior.
- Servicio cartográfico: prever selección manual de punto y persistencia local temporal si el mapa externo falla.
- Privacidad: limitar el acceso a información de contacto y ubicación residencial.
- Integridad territorial: validar que barrios, comunas y corregimientos correspondan a la división oficial utilizada por la entidad.
- Trazabilidad: registrar cambios relevantes y conservar quién creó o modificó cada dato.

La siguiente iteración puede abordar la configuración estática de puntos de atención, la generación de hojas de ruta, asignación de cargas por encuestador y seguimiento al límite de 15 días, según la priorización del Product Owner.

12. Conclusiones

Los cuatro primeros requerimientos establecen la base operativa del sistema: identidad y permisos, captura estructurada de solicitudes, ubicación geográfica y consulta consolidada para enrutamiento. La solución reduce la dependencia de hojas de cálculo aisladas y permite construir trazabilidad desde el momento en que se registra la solicitud.

El modelo de datos propuesto separa usuarios, roles, territorio, puntos de atención, solicitudes, coordenadas y eventos, lo que favorece integridad y evolución del sistema. Scrum permite organizar la entrega en incrementos verificables y DevOps aporta control de cambios y repetibilidad.

Para cerrar la entrega académica con evidencia completa, el equipo debe contrastar este informe con el código real de la rama, sustituir las tecnologías propuestas por las efectivamente utilizadas y completar las bitácoras con commits, ejecuciones y capturas auténticas.

13. Referencias

Causimanse, J. R. (s. f.). Proyecto – Desarrollo de Software II. Documento de requerimientos suministrado por el curso. Secciones: Contexto Operativo, Logística, Operación Actual y requerimientos de la aplicación web.

Repositorio del proyecto: LuisdaUV, Dataciudad, rama entrega-1-puntos-1-4. https://github.com/LuisdaUV/Dataciudad/tree/entrega-1-puntos-1-4 (referencia suministrada por el equipo; contenido de la rama pendiente de verificación en esta sesión).

Schwaber, K., & Sutherland, J. (2020). The Scrum Guide. https://scrumguides.org/

Anexo A. Distribución sugerida de responsabilidades

Integrante	Responsabilidad sugerida	Evidencia que debe aportar
Luis David Rico	Coordinación de integración y autenticación/roles.	Commits, pruebas de login y matriz de permisos.
Sebastián Reyes	Registro de solicitudes y validaciones territoriales.	Commits, captura del formulario y pruebas de validación.
Felipe Torres	Georreferenciación, bitácora y documentación técnica.	Commits, captura del mapa/bitácora y pruebas.

La asignación anterior es una propuesta para organizar el informe. Debe reemplazarse si la distribución real del trabajo fue diferente.